

CYBERSECURITY & ETHICAL HACKING INTERNSHIP

# Task 2 — Lab Build Report

*Building a Personal, Isolated Cybersecurity Practice Lab*

|                  |                                                           |
|------------------|-----------------------------------------------------------|
| PREPARED BY      | Jenil Patel                                               |
| REPORT DATE      | 13 July 2026                                              |
| LAB COMPONENTS   | Kali Linux (Attacker VM) + OWASP Juice Shop (Target)      |
| VIRTUALIZATION   | Oracle VirtualBox                                         |
| NETWORK DESIGN   | NAT (Internet/Updates) + Host-Only (Isolated Lab Traffic) |
| VALIDATION TOOLS | Nmap, Wireshark, Burp Suite                               |

# Table of Contents

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>1. Objective.....</b>                             | <b>3</b>  |
| 1.1 What This Task Delivers.....                     | 3         |
| 1.2 Scope and Boundaries.....                        | 3         |
| <b>2. Lab Architecture.....</b>                      | <b>3</b>  |
| 2.1 NAT vs. Host-Only — Design Comparison.....       | 4         |
| <b>3. Prerequisites.....</b>                         | <b>4</b>  |
| 3.1 Host System Requirements.....                    | 4         |
| 3.2 Software and Tools Used.....                     | 4         |
| 3.3 Risk and Isolation Considerations.....           | 5         |
| <b>4. Build Methodology.....</b>                     | <b>5</b>  |
| <b>5. Environment Setup.....</b>                     | <b>6</b>  |
| 5.1 Attacker Machine — Kali Linux VM.....            | 6         |
| 5.2 Network Adapter Configuration.....               | 6         |
| 5.3 Verifying Interfaces on the Kali VM.....         | 6         |
| <b>6. Target Deployment — OWASP Juice Shop.....</b>  | <b>7</b>  |
| <b>7. Connectivity Validation — Nmap.....</b>        | <b>8</b>  |
| <b>8. Traffic Analysis — Wireshark.....</b>          | <b>9</b>  |
| <b>9. Web Traffic Interception — Burp Suite.....</b> | <b>9</b>  |
| <b>10. Evidence Summary.....</b>                     | <b>10</b> |
| 10.1 Virtual Machine Inventory.....                  | 10        |
| 10.2 IP / Network Configuration.....                 | 10        |
| 10.3 Validation Checklist.....                       | 10        |
| 10.4 Screenshot Index.....                           | 11        |
| <b>11. Challenges and Observations.....</b>          | <b>11</b> |
| <b>12. Recommendations and Next Steps.....</b>       | <b>11</b> |
| <b>13. References and Learning Resources.....</b>    | <b>12</b> |
| <b>14. Conclusion.....</b>                           | <b>12</b> |
| 14.1 Key Takeaways.....                              | 12        |
| <b>15. Appendix — Deliverables.....</b>              | <b>13</b> |

# 1. Objective

The objective of this task was to design and build a safe, fully isolated cybersecurity practice environment on a personal computer. This hands-on lab forms the foundation for all subsequent tasks in the internship track, including vulnerability assessment, web application testing, exploitation practice, and incident response.

The lab was built with the following components:

- One Attacker machine — Kali Linux
- One vulnerable target application — OWASP Juice Shop, deployed via Docker
- Proper network segmentation using NAT (for internet access/updates) and a Host-Only network (for isolated lab traffic)

## 1.1 What This Task Delivers

| Skill Area        | Practical Outcome                                 |
|-------------------|---------------------------------------------------|
| Virtualization    | Running multiple isolated systems for pentesting  |
| Lab isolation     | Preventing real-world system damage               |
| Web targets setup | Deploying an intentionally vulnerable application |
| Basic networking  | NAT vs. Host-Only and subnet usage                |
| Tool awareness    | Nmap, Burp Suite and Wireshark validation         |

## 1.2 Scope and Boundaries

To keep the build focused and reproducible, the following scope was applied:

- In scope: local virtualization, dual-network configuration, target deployment, and validation using Nmap, Wireshark, and Burp Suite.
- Out of scope: any scanning, exploitation, or traffic capture directed at systems outside the lab's own Host-Only network.
- The target application, OWASP Juice Shop, was chosen because it is intentionally vulnerable and explicitly designed for this kind of legal, controlled practice.

# 2. Lab Architecture

The lab uses a two-network design so that the attacker and target machines can communicate with each other on a private, host-only segment, while still retaining a separate NAT-based path to the internet for patching and package installation. This mirrors the segmentation used in real penetration-testing labs, where test traffic never touches the production or public network.

```
+-----+
| Your Computer (Host System) |
|                               |
| +-- Virtualization Software  |
|   |-- Kali Linux   (Attacker) |
|   +-- Juice Shop   (Target)   |
|                               |
| Networks:                   |
|                               |
+-----+
```

```
|   - NAT      -> Internet / Updates   |
|   - Host-Only -> Isolated Lab Traffic|
+-----+

```

On the deployed lab, the Kali Linux attacker VM was configured with two network adapters — Adapter 1 attached to NAT for outbound internet access, and Adapter 2 attached to a Host-Only network (vboxnet0) for direct, isolated communication with the target application. Both were confirmed active and connected in the VirtualBox settings shown in Section 5.2.

This separation is deliberate. If the attacker VM used only NAT, its lab traffic would be routed and potentially exposed alongside normal internet traffic; if it used only Host-Only networking, it would have no route out to fetch updates or new tools. Running both adapters together gives the attacker machine a private, addressable lab segment for testing while still behaving like a normal, patchable Linux system.

## 2.1 NAT vs. Host-Only — Design Comparison

| Aspect                         | NAT Adapter                              | Host-Only Adapter                      |
|--------------------------------|------------------------------------------|----------------------------------------|
| Internet access                | Yes — used for updates/downloads         | No — fully isolated                    |
| Visible to the host network    | No — traffic is translated by VirtualBox | No — private virtual switch only       |
| Attacker ↔ Target reachability | Not used for lab traffic                 | Yes — primary channel for testing      |
| Risk if misused                | Low, standard outbound traffic           | None — cannot reach production systems |
| Used in this lab for           | apt updates, tool downloads              | Nmap, Wireshark, Burp Suite traffic    |

## 3. Prerequisites

Before beginning the build, the following host system requirements and prerequisites were reviewed to ensure the lab would run smoothly:

### 3.1 Host System Requirements

| Requirement      | Recommendation                        |
|------------------|---------------------------------------|
| RAM              | Minimum 8 GB, preferably 16 GB        |
| Disk Space       | 40–60 GB free                         |
| CPU              | Virtualization (VT-x / AMD-V) enabled |
| Operating System | Windows / macOS / Linux               |

### 3.2 Software and Tools Used

| Tool              | Purpose                                  |
|-------------------|------------------------------------------|
| Oracle VirtualBox | Virtualization platform hosting both VMs |

| Tool             | Purpose                                                |
|------------------|--------------------------------------------------------|
| Kali Linux VM    | Attacker machine with pre-installed security tools     |
| OWASP Juice Shop | Intentionally vulnerable web application (target)      |
| Docker           | Containerized deployment of Juice Shop                 |
| Nmap             | Host discovery and connectivity validation             |
| Wireshark        | Network traffic capture and packet analysis            |
| Burp Suite       | Web proxy for intercepting and inspecting HTTP traffic |

### 3.3 Risk and Isolation Considerations

Because the target application is intentionally vulnerable, a few precautions were treated as mandatory rather than optional before any testing traffic was sent:

- The target container is only ever reachable from the Host-Only network — it is never exposed on the NAT adapter or on the physical host network.
- No port-forwarding rules were created that would expose the Juice Shop container to the internet.
- The attacker VM's snapshots are taken before any exploitation activity, so the lab can be reset to a clean state if needed.
- All scanning and interception in this report targeted only machines inside the lab's own 192.168.56.0/24 subnet.

## 4. Build Methodology

The lab was assembled in five sequential stages, following the step-by-step instructions provided in the task brief. Each stage was validated before moving on to the next, so that any misconfiguration could be isolated and corrected early rather than compounding later in the build.

| Stage | Action                                                           | Outcome                                                                  |
|-------|------------------------------------------------------------------|--------------------------------------------------------------------------|
| 1     | Install VirtualBox; create a Host-Only network; enable NAT       | Virtualization platform ready with both required network types available |
| 2     | Deploy the Kali Linux attacker VM (2 CPUs, 3–4 GB RAM, dual NIC) | Attacker VM boots with NAT + Host-Only adapters attached                 |
| 3     | Deploy the target (OWASP Juice Shop via Docker)                  | Vulnerable web application running and exposed on port 3000              |
| 4     | Verify connectivity (`nmap -sn` across the lab subnet)           | Attacker machine confirmed to see the target on the isolated network     |
| 5     | Basic validation (browser access, Burp interception, Nmap scan)  | End-to-end tooling confirmed functional against the target               |

Working folders were also created on the attacker VM to keep scan output, notes, packet captures and screenshots organised for this and future tasks:

```
sudo apt update && sudo apt upgrade -y
mkdir -p ~/lab/{scans,notes,pcaps,screenshots}
```

## 5. Environment Setup

### 5.1 Attacker Machine — Kali Linux VM

The Kali Linux attacker virtual machine was imported and deployed inside Oracle VirtualBox as an ARM64 guest, allocated its own virtual disk, VirtioSCSI storage controller, and dedicated display and audio devices. The completed VM configuration (left) confirms the guest OS, storage, and dual network adapters. Once powered on, the desktop environment (right) loaded successfully, confirming the VM boots correctly and is ready for tool usage.

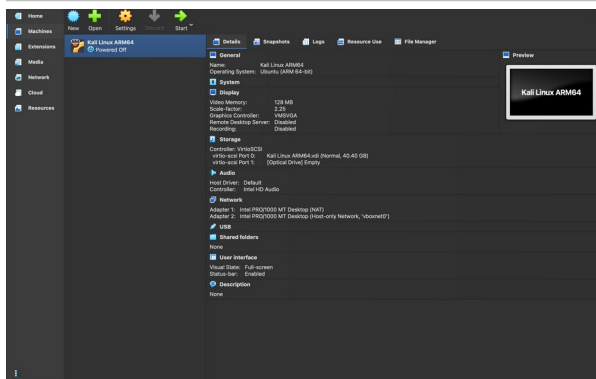


Figure 1 — VirtualBox machine summary for the Kali Linux ARM64 attacker VM.



Figure 2 — Kali Linux desktop running inside VirtualBox.

### 5.2 Network Adapter Configuration

Two virtual network adapters were configured on the attacker VM in line with the required NAT + Host-Only design: Adapter 1 was attached to NAT, providing outbound internet access for updates and package installation, while Adapter 2 was attached to a Host-Only network (vboxnet0), providing an isolated segment for direct communication with the target machine.

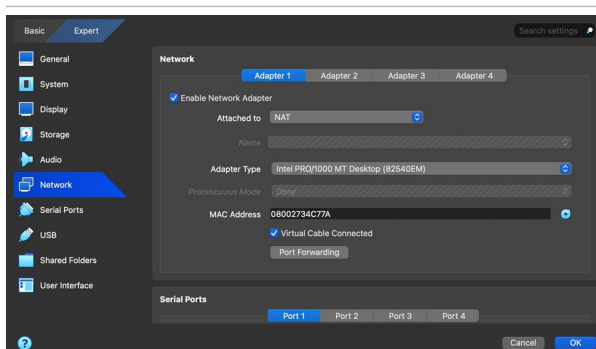


Figure 3 — Adapter 1 configured with NAT.

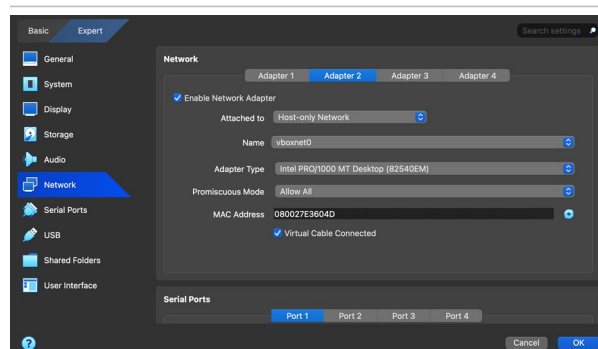


Figure 4 — Adapter 2 configured with Host-Only (vboxnet0).

### 5.3 Verifying Interfaces on the Kali VM

Inside the running Kali Linux VM, the `ip a` command was used to confirm both network interfaces were active and correctly addressed — the NAT interface (eth0) received a 10.0.2.15/24 address, while the Host-Only interface (eth1) was also up, alongside the Docker bridge interface (docker0) used to expose the target container.

```

jenil@kali: ~
Session Actions Edit View Help
└─(jenil@kali)-[~]
   └─$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
8e15db5f8579   bkimminich/juice-shop   "/nodejs/bin/node /j-  13 seconds ago   Up 13 seconds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   juice-shop

└─(jenil@kali)-[~]
   └─$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
8e15db5f8579   bkimminich/juice-shop   "/nodejs/bin/node /j-  2 minutes ago   Up 2 minutes    0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   juice-shop

└─(jenil@kali)-[~]
   └─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:34:c7:7a brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 84561sec preferred_lft 64561sec
    inet6 fd17:625c:f037:2:cd86:5610:c063:efcc/64 scope global temporary dynamic
        valid_lft 86228sec preferred_lft 14228sec
    inet6 fd17:625c:f037:2:a00:27ff:fe34:c77a/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86228sec preferred_lft 14228sec
    inet6 fe80::a00:27ff:fe34:c77a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:34:c7:7a brd ff:ff:ff:ff:ff:ff
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 66:f9:fc:2e:57:aa brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::64f9:fcff:fe2e:57aa/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
5: veth@f282501f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether bc:8c:18:12:aa:65 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::bc8c:18ff:fe12:aa65/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever

└─(jenil@kali)-[~]

```

Figure 5 — Output of `docker ps` and `ip a`, confirming the Juice Shop container is running and both network interfaces are active.

## 6. Target Deployment — OWASP Juice Shop

OWASP Juice Shop was selected as the vulnerable target application. It was deployed as a Docker container directly inside the Kali Linux VM using the official image, exposing the application on port 3000:

```

sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo docker pull bkimminich/juice-shop
sudo docker run -d -p 3000:3000 --name juice bkimminich/juice-shop

```

The `docker ps` output (Figure 5) confirmed the container was up and running, with port 3000 mapped and available on both IPv4 and IPv6. Accessing the application from a browser on port 3000 confirmed the target was successfully deployed and reachable, with the storefront loading correctly, including its product catalogue and tracking-cookie banner.

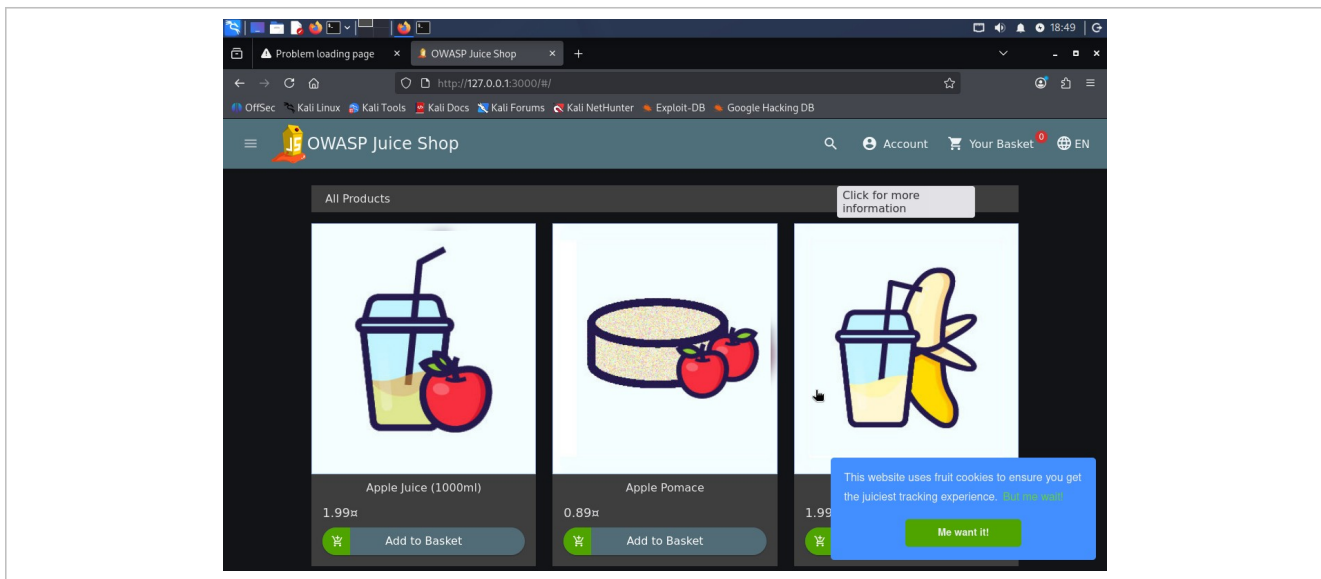


Figure 6 — OWASP Juice Shop storefront running successfully at `http://127.0.0.1:3000/`.

## 7. Connectivity Validation — Nmap

To confirm that the attacker machine could reach the target across the Host-Only network, a host-discovery sweep was run against the lab subnet:

```
nmap -sn 192.168.56.0/24
```

The scan reported all 256 addresses in the 192.168.56.0/24 range as up, with sub-millisecond latency — confirming the Host-Only network was correctly configured and that the attacker VM could see every host on the isolated lab segment, including the target machine.

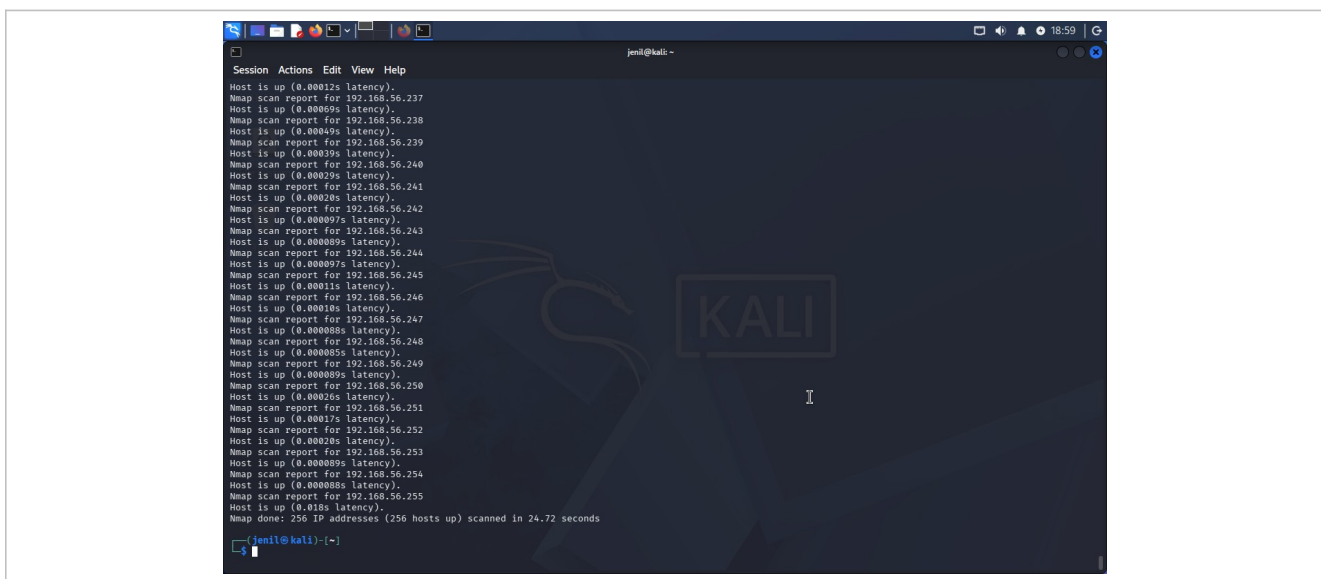


Figure 7 — Nmap host-discovery scan confirming 256 hosts reachable on the 192.168.56.0/24 Host-Only network.



## 8. Traffic Analysis — Wireshark

Wireshark was launched on the eth1 (Host-Only) interface to validate that lab network traffic could be captured for later analysis. The tool was confirmed to start and stop capture sessions correctly, writing capture files to /tmp for further review.

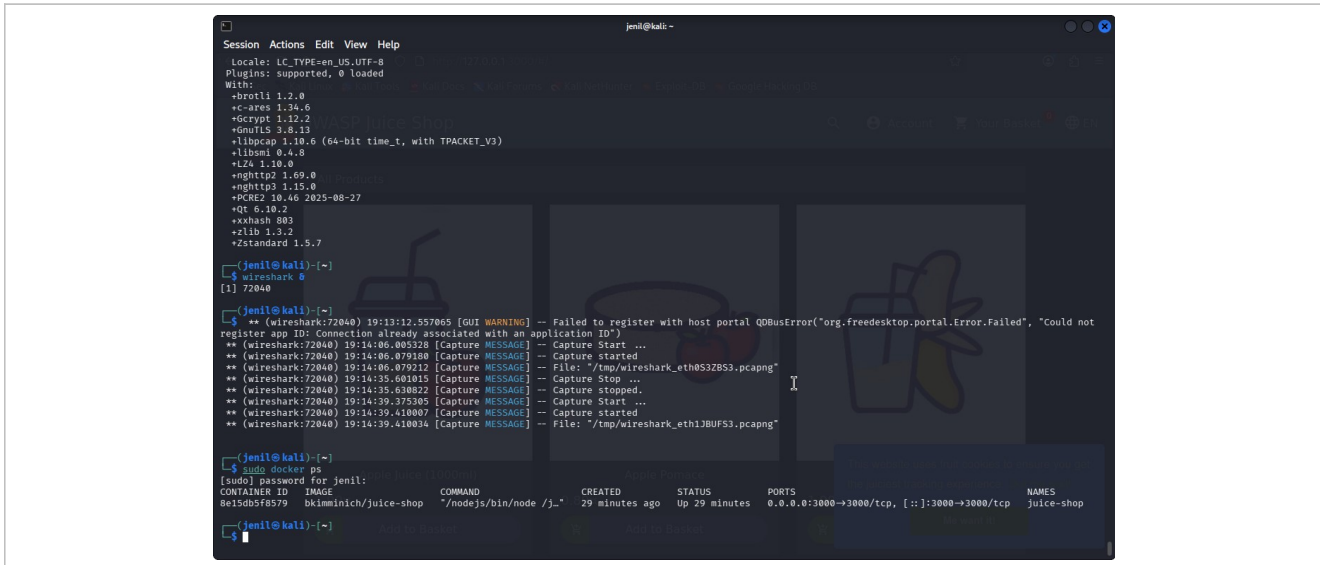


Figure 8 — Wireshark launched from the terminal, showing successful capture start/stop events and the Juice Shop container status.

A short initial capture on eth1 (left) picked up multicast DNS (mDNS) query and response traffic. The capture was then left running for a longer interval, gathering 61 packets (right) including additional MDNS and ICMPv6 Router Advertisement traffic — confirming stable, continuous packet capture on the lab network.

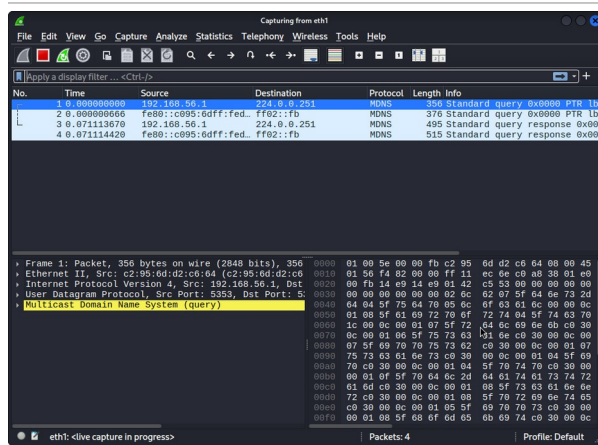


Figure 9 — Initial capture on eth1 (4 packets, MDNS).

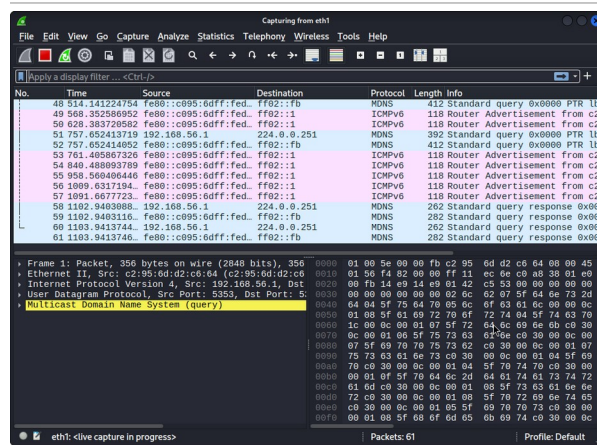


Figure 10 — Extended capture on eth1 (61 packets).

## 9. Web Traffic Interception — Burp Suite

Burp Suite Community Edition was configured to validate the web-proxy interception workflow that will be used in later web-application testing tasks. A proxy listener was configured on 127.0.0.1:8080, with interception rules enabled to stall requests for review, excluding common static file types (images, CSS, fonts, etc.). With the listener

active, browser traffic routed through the proxy was captured in the HTTP History tab, confirming successful interception and logging of outbound requests.

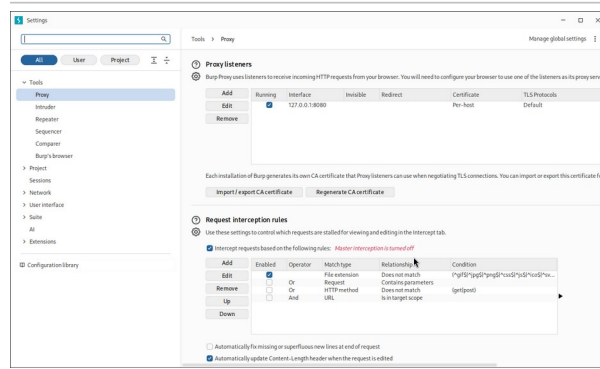


Figure 11 — Burp Suite Proxy listener on 127.0.0.1:8080 and interception rules.

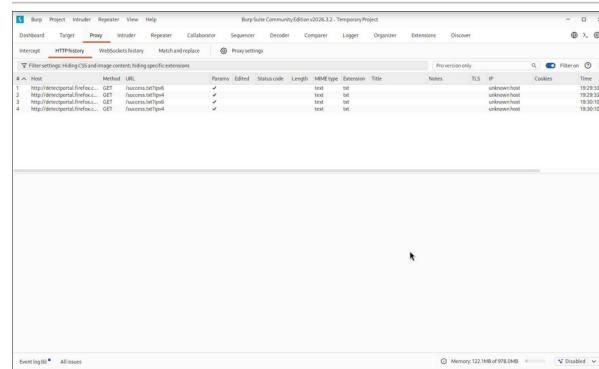


Figure 12 — Burp Suite HTTP History showing intercepted requests.

## 10. Evidence Summary

### 10.1 Virtual Machine Inventory

| VM / Component   | Role               | OS / Image                          | Status               |
|------------------|--------------------|-------------------------------------|----------------------|
| Kali Linux ARM64 | Attacker           | Kali Linux (Ubuntu ARM 64-bit base) | Running              |
| juice-shop       | Target (container) | bkimminich/juice-shop               | Up, port 3000 mapped |

### 10.2 IP / Network Configuration

| Interface | Network              | Address         | Purpose                                 |
|-----------|----------------------|-----------------|-----------------------------------------|
| eth0      | NAT                  | 10.0.2.15/24    | Internet access & package updates       |
| eth1      | Host-Only (vboxnet0) | 192.168.56.x/24 | Isolated attacker-to-target lab traffic |
| docker0   | Docker bridge        | 172.17.0.1/16   | Container networking for Juice Shop     |

### 10.3 Validation Checklist

| Validation Step                         | Tool       | Result                       |
|-----------------------------------------|------------|------------------------------|
| Host-discovery across lab subnet        | Nmap (-sn) | 256/256 hosts reachable      |
| Target application reachable in browser | Firefox    | Juice Shop storefront loaded |

| Validation Step                       | Tool       | Result                              |
|---------------------------------------|------------|-------------------------------------|
| Packet capture on Host-Only interface | Wireshark  | 61 packets captured (MDNS / ICMPv6) |
| Proxy interception of browser traffic | Burp Suite | Requests logged in HTTP History     |

## 10.4 Screenshot Index

| Figure  | Description                                                              |
|---------|--------------------------------------------------------------------------|
| 1 – 2   | Kali Linux ARM64 VM summary and running desktop                          |
| 3 – 4   | NAT (Adapter 1) and Host-Only (Adapter 2) network configuration          |
| 5       | `docker ps` and `ip a` output confirming interfaces and container status |
| 6       | OWASP Juice Shop storefront reachable in the browser                     |
| 7       | Nmap host-discovery sweep across the lab subnet                          |
| 8       | Wireshark launched and capturing on eth1                                 |
| 9 – 10  | Initial (4-packet) and extended (61-packet) Wireshark captures           |
| 11 – 12 | Burp Suite proxy listener configuration and HTTP History                 |

## 11. Challenges and Observations

- Ensuring the Host-Only adapter (vboxnet0) was enabled and had a virtual cable connected before boot was necessary for the attacker VM to see the target network.
- Running Docker commands required elevated privileges (sudo), so the Juice Shop container was deployed and inspected using an authenticated shell session.
- Wireshark needed to be pointed explicitly at the eth1 interface to capture Host-Only lab traffic rather than the NAT-facing eth0 interface.
- Burp Suite's default interception rule excludes common static file extensions, which kept the HTTP History focused on meaningful application requests.
- The initial Nmap sweep took under 25 seconds to enumerate a full /24, which was fast enough to re-run after any network change to re-confirm reachability.
- Because both VM adapters share the same physical host, the ARM64 Kali image needed the VirtioSCSI storage controller for stable disk performance during setup.

## 12. Recommendations and Next Steps

With the base lab validated, the following actions are recommended before moving into the next task in the internship track:

- Take a VirtualBox snapshot of both the attacker VM and the target container's known-good state, so the environment can be reset quickly after a failed exploitation attempt.
- Import the Burp Suite CA certificate into the browser used for testing, to allow HTTPS interception in addition to the HTTP traffic captured in Section 9.

- Extend the Nmap validation from a host-discovery sweep (`-sn`) to a service/version scan (`-sV -sC`) against the target once vulnerability assessment begins.
- Save Wireshark captures directly into the `~/lab/pcaps` working folder created in Section 4, using descriptive filenames tied to each testing session.
- Keep the Host-Only network as the only path used for actual attack traffic against the target, reserving the NAT adapter strictly for updates.

## 13. References and Learning Resources

The following official documentation and beginner-friendly resources were used as references while building and validating this lab:

| Resource                                                                                                                               | Purpose                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| VirtualBox — <a href="https://www.virtualbox.org/wiki/Downloads">virtualbox.org/wiki/Downloads</a>                                     | Official download and setup for the virtualization platform            |
| Kali Linux VM — <a href="https://kali.org/get-kali">kali.org/get-kali</a>                                                              | Official prebuilt Kali Linux virtual machine images                    |
| OWASP Juice Shop — <a href="https://github.com/juice-shop/juice-shop">github.com/juice-shop/juice-shop</a>                             | Source and documentation for the target application                    |
| Docker Engine — <a href="https://docs.docker.com/engine/install">docs.docker.com/engine/install</a>                                    | Installation guide for the container runtime used to deploy Juice Shop |
| Burp Suite Documentation — <a href="https://portswigger.net/burp/documentation/desktop">portswigger.net/burp/documentation/desktop</a> | Reference for proxy configuration and interception rules               |
| Wireshark Documentation — <a href="https://www.wireshark.org/docs">wireshark.org/docs</a>                                              | Reference for packet capture and protocol analysis                     |

## 14. Conclusion

This task successfully established a safe, segmented, and reproducible personal cybersecurity lab. A Kali Linux attacker VM and an OWASP Juice Shop target container were deployed inside Oracle VirtualBox, connected through a dual NAT + Host-Only network design that keeps lab traffic fully isolated from the host's production network while still allowing internet access for updates.

Connectivity and tooling were validated end-to-end: Nmap confirmed host discovery across the lab subnet, the Juice Shop application was reachable and functional in the browser, Wireshark successfully captured live traffic on the isolated interface, and Burp Suite was configured and verified as an intercepting proxy. With this foundation in place, the environment is ready to support the subsequent tasks in the internship track, including vulnerability assessment, web application testing, exploitation practice, and incident response.

### 14.1 Key Takeaways

- A dual-adapter (NAT + Host-Only) design is sufficient to keep a vulnerable target fully isolated while still allowing the attacker VM to stay updated.
- Docker significantly simplifies target deployment — the Juice Shop application was up and reachable within minutes of installing the Docker engine.
- Validating each layer independently — network reachability (Nmap), packet visibility (Wireshark), and proxy interception (Burp Suite) — makes it much easier to isolate problems before more advanced testing begins.

## 15. Appendix — Deliverables

---

The following deliverables were produced as part of this task:

- Full Lab Build report (this document), including annotated screenshots
- Evidence folder containing:
  - VM list
  - IP configuration
  - Ping / Nmap validation output
  - Burp Suite and Wireshark screenshots

---

— End of Report —